

Data Mastery 120: Roadmap Integral de Minería y Análisis de Datos

Día 5: Seguridad y Entorno

3.1.5 Semana 1 - Día 5: Seguridad y Entorno

"Protección de Activos y Control de Acceso (RBAC)"

Has certificado la **calidad de los datos** en el **Día 4**. Ahora, para cerrar la primera semana, debemos **blindar el entorno**. En el mundo del **análisis de datos** de 2026, la **Seguridad de la Información** no es opcional; es la base de la **ética profesional**. Hoy aprenderás a **proteger tus credenciales**, **gestionar accesos** en la **base de datos** y asegurar que tus **reportes** en **Power BI** sean **confidenciales**.

El **objetivo** de hoy es asegurar que solo las **personas (o procesos) autorizados** puedan interactuar con el **dataset** de **IBM HR**, eliminando cualquier rastro de **contraseñas** en el **código fuente**.

Diccionario Técnico Operativo

- 'Environment Variables' (Variables de Entorno):
 - **Valores dinámicos** que se almacenan en el **sistema operativo**. Permiten que una **aplicación** se adapte a **desarrollo**, **pruebas** o **producción** sin modificar el **código fuente**, mejorando la **gestión** y la **seguridad**.
 - **Sintaxis General en Python**:
 - Para manejar **Variables de Entorno** (Environment Variables) de forma segura, se suele instalar la **librería python-dotenv** para cargarlas desde un **archivo externo** y el **módulo nativo os** para leerlas.
 - **Instalación**: `pip install python-dotenv` .
 - **Carga**: Busca un archivo llamado **.env** y carga su contenido.

```
from dotenv import load_dotenv  
load_dotenv()
```

- **Lectura**: Recupera el **valor** asociado a esa **Variable de Entorno** (Environment Variable).

```
import os  
value = os.getenv('DB_PASS')
```

- **Caso Práctico**: Almacenar la **contraseña** de **PostgreSQL** en un archivo **.env** que **Python** lee pero **Git** ignora.

- Crea un **archivo** llamado **.env** en la **raíz de tu proyecto** con tus **credenciales reales**.

```
DB_HOST = localhost
DB_NAME = hr_analytics
DB_USER = postgres
DB_PASS = MiPasswordSeguro123
```

- **Pitfall:**

- EL **error más crítico** al usar **Variables de Entorno (Environment Variable)** es olvidar el **archivo .env** al **archivo .gitignore**.
- **Riesgo:** Si no incluyes el **archivo .env** en tu **archivo .gitignore**, al hacer un **git push**, tu **contraseña** de **PostgreSQL** y otras **claves** se publicarían en **GitHub** para que cualquiera las vea.
- **Solución:** Crea un **archivo .gitignore** y añade simplemente la **línea .env** antes de tu **primer commit**.

- **'RBAC' (Role-Based Access Control):**

- Metodología de **seguridad** que asigna **permisos** (ej. leer, actualizar, borrar) a **grupos** o **roles** en lugar de a **usuarios individuales**.
- **Sintaxis (SQL):**
 - **CREATE ROLE** nombre_rol; — Crea el **contenedor de permisos**.
 - **GRANT** privilegio **ON** objeto **TO** nombre_rol; — **Asigna permisos al rol**.
 - **CREATE USER** nombre_usuario ...; — Crea al **usuario**.
 - **GRANT** nombre_rol **TO** nombre_usuario; — **Asigna el rol al usuario**.
- **Caso Práctico:**
 - Crear un **perfil** que solo pueda hacer **SELECT** para evitar que un usuario de **Power BI** borre accidentalmente la tabla de empleados.

```
/*****
*****/
```

Título: Configuración de Acceso para Analista BI

Objetivo: Crear un usuario específico para análisis de datos con permisos limitados

de lectura sobre la tabla de empleados.

Descripción: Este script automatiza la creación de un rol de seguridad, asigna permisos de lectura (SELECT) a dicho rol, crea un nuevo usuario y lo asocia al rol para garantizar acceso seguro.

Archivo SQL: day05_create_bi_analyst_role_and_user.sql

Archivo PNG: day05_create_bi_analyst_role_and_user_01.png
day05_create_bi_analyst_role_and_user_02.png
day05_create_bi_analyst_role_and_user_03.png
day05_create_bi_analyst_role_and_user_04.png

```
*****  
*****/
```

-- 1. Crear el rol (perfil) que agrupará los permisos de BI

CREATE ROLE bi_analyst_role;

-- 2. Asignar permiso de lectura (SELECT) sobre la tabla de empleados a este rol

GRANT SELECT ON employee_master_data **TO** bi_analyst_role;

-- 3. Crear el usuario físico que utilizará el analista

CREATE USER bi_user **WITH PASSWORD** 'password_seguro';

-- 4. Asignar el rol creado al usuario (hereda los permisos de lectura)

GRANT bi_analyst_role **TO** bi_user;

- Resultado PNG: day05_create_bi_analyst_role_and_user_01.png

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,008s
Start time	Sun May 10 17:27:04 CST 2026
Finish time	Sun May 10 17:27:04 CST 2026
Query	-- 1. Crear el rol (perfil) que agrupará los permisos de BI CREATE ROLE bi_analyst_role

- Resultado PNG: day05_create_bi_analyst_role_and_user_02.png

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,005s
Start time	Sun May 10 17:31:44 CST 2026
Finish time	Sun May 10 17:31:44 CST 2026
Query	-- 2. Asignar permiso de lectura (SELECT) sobre la tabla de empleados a este rol GRANT SELECT ON employee_master_data TO bi_analyst_role

- Resultado PNG: day05_create_bi_analyst_role_and_user_03.png

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,058s
Start time	Sun May 10 17:32:37 CST 2026
Finish time	Sun May 10 17:32:37 CST 2026
Query	-- 3. Crear el usuario físico que utilizará el analista CREATE USER bi_user WITH PASSWORD 'password_seguro'

- Resultado PNG: day05_create_bi_analyst_role_and_user_04.png

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,007s
Start time	Sun May 10 17:33:19 CST 2026
Finish time	Sun May 10 17:33:19 CST 2026
Query	-- 4. Asignar el rol creado al usuario (hereda los permisos de lectura) GRANT bi_analyst_role TO bi_user

- Resultado: El **bi_user** podrá usar **SELECT * FROM employee_master_data** en **Power BI**, pero si intenta ejecutar **DELETE**, **DROP** o **UPDATE**, SQL denegará la acción.
- Pitfall:
 - Dar permisos de **SUPERUSER** a una aplicación que solo necesita **leer datos**.

```
/******  
*****
```

Título: Corrección de Permisos de Usuario BI

Objetivo: Elevar los privilegios del usuario 'BI_User' a superusuario.

Descripción: Este script otorga permisos totales al usuario encargado de Business

Intelligence para asegurar el acceso a todos los datos.

Archivo SQL: day05_alter_BIUser_permissions.sql

```
*****  
*****/
```

```
-- [BLOQUE: ELEVACIÓN DE PRIVILEGIOS]
```

```
-- Se modifica el usuario 'BI_User' para otorgarle permisos de  
superusuario (administrador total).
```

```
-- ¡ERROR CRÍTICO! - Nota: Esto es un riesgo de seguridad alto y  
debe evitarse en producción.
```

```
ALTER USER bi_user WITH SUPERUSER;
```

```
-- [BLOQUE: ALTERNATIVA DE PERMISOS]
```

```
-- Alternativa para sistemas MySQL/MariaDB: Otorga todos los  
privilegios sobre todas las bases de datos.
```

```
-- GRANT ALL PRIVILEGES ON *.* TO 'BI_User';
```

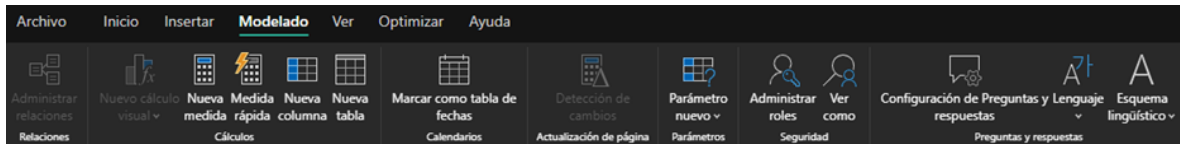
- **Riesgo:** Si la app **Power BI** es comprometida o si el analista comete un error, puede ejecutar un **DROP TABLE** o **DELETE** y borrar toda la base de empleados, ya que los permisos de **SUPERUSER** anulan todas las restricciones de seguridad.
- **'Row-Level Security' (RLS - Seguridad a Nivel de Fila):**
 - **Filtro de seguridad** que restringe el acceso a los datos a nivel de registros (**filas**) para controlar de forma segura lo que los usuarios pueden ver según sus **atributos** (ej. su departamento).
 - **Sintaxis (DAX):**

```
[NombreDeLaTabla].[NombreDeColumna] = "ValorAFiltrar"
```
- **Estática:** [Department] = "Sales"

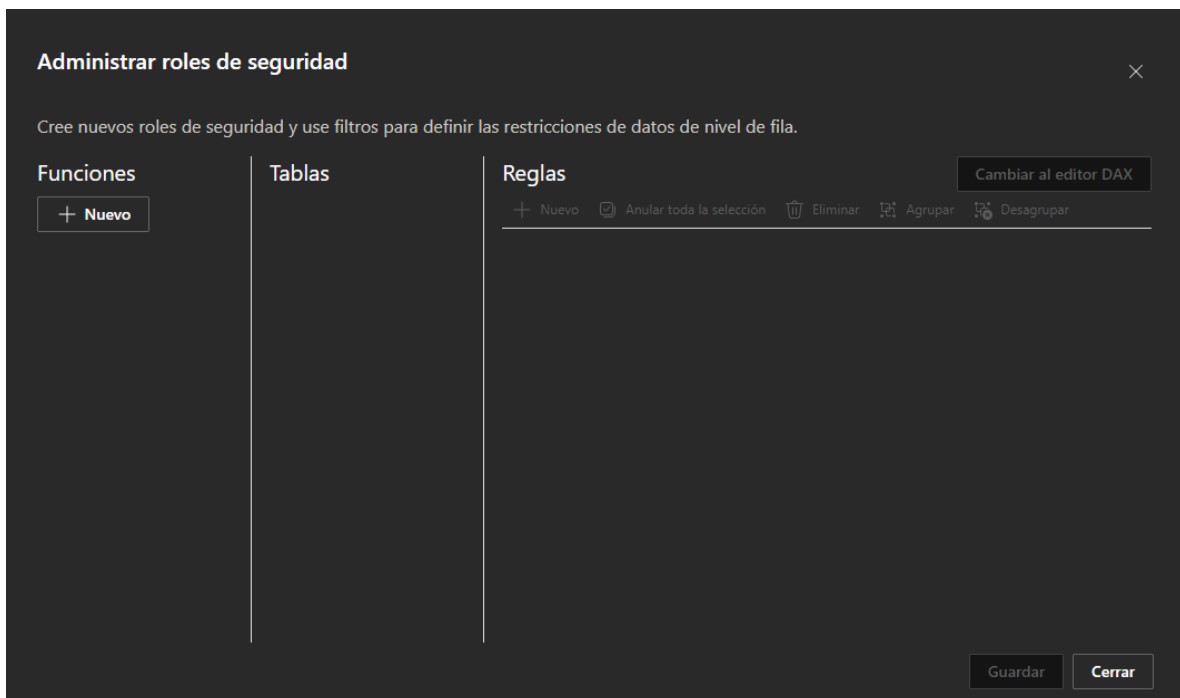
- **Dinámica / Segura:** [Email] = USERPRINCIPALNAME() (Filtra por el correo del usuario que inició sesión).
- **Caso Práctico:** Configurar **Power BI** para que el gerente de **Ventas** solo vea los datos de su equipo y no los de **Recursos Humanos** .

Pasos en **Power BI Desktop**:

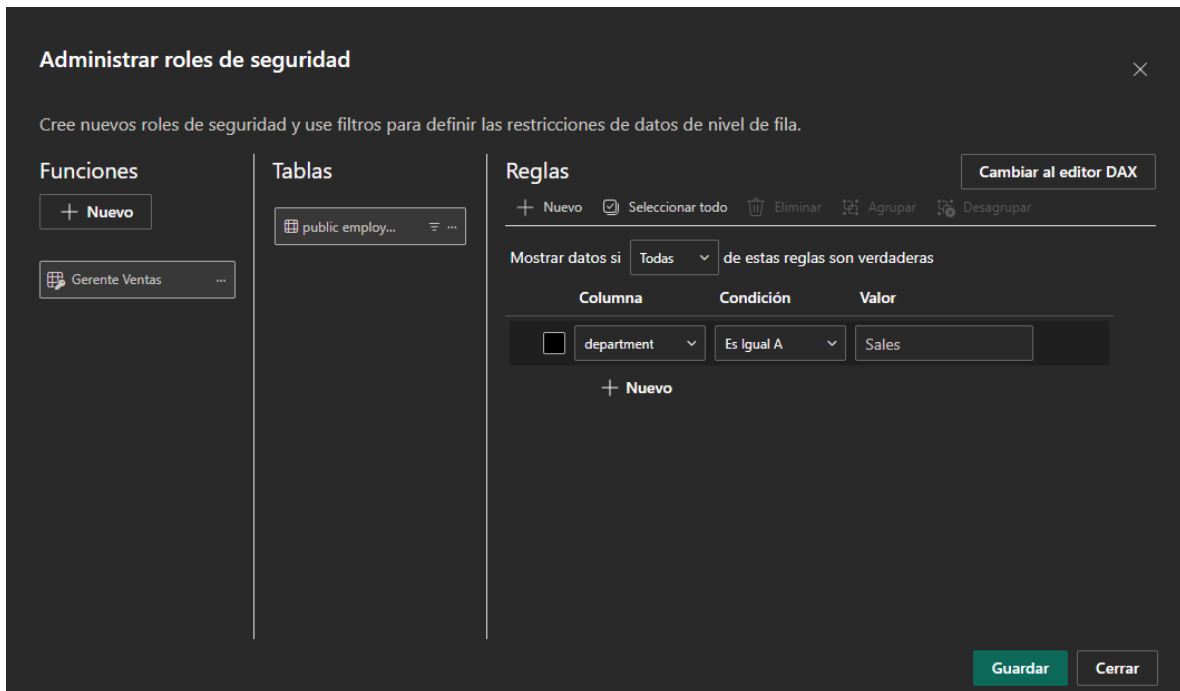
- Vaya a la pestaña **Modelado (Modeling)** > **Administrar roles (Manage Roles)**.



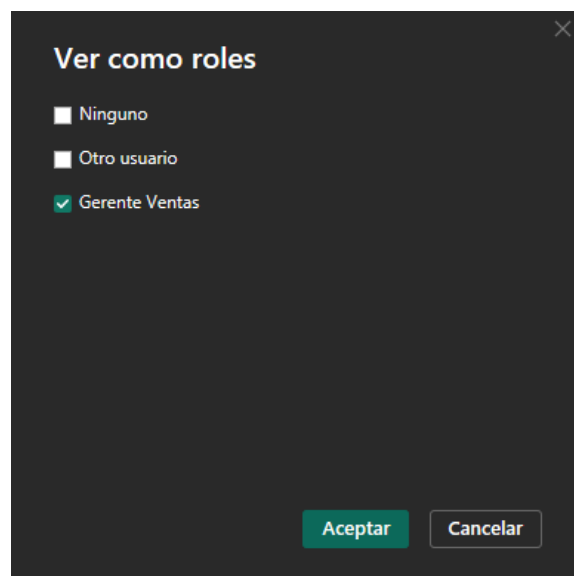
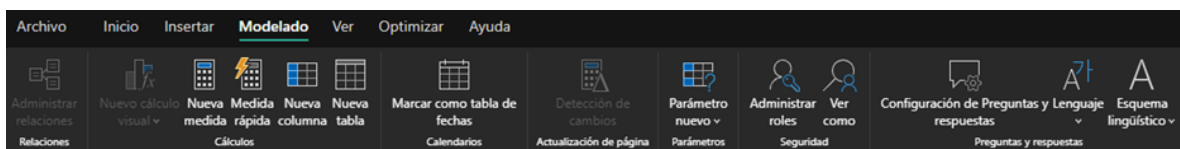
- Haga clic en **Crear** y nombre el rol como **Gerente_Ventas**.



- En la tabla **HR Analytics**, seleccione la columna **Department**.
- En la expresión **DAX**, escriba: **[Department] = "Sales"**.
- Haga clic en **Guardar**.



- **Validación:** Haga clic en **Ver como (View as)** y seleccione **Gerente_Ventas**. El informe mostrará solo empleados de **Ventas**.



employee_number	Primera fecha: department	Primera fecha: job_role	Suma de monthly_income
38	Sales	Manager	18947
323	Sales	Manager	19517
329	Sales	Manager	19068
558	Sales	Manager	18041
992	Sales	Manager	17444
1029	Sales	Manager	18303
1038	Sales	Manager	19845
1045	Sales	Manager	17650
1124	Sales	Manager	17465
1204	Sales	Manager	17048
1267	Sales	Manager	18213
1277	Sales	Manager	18824
Total	Sales	Manager	353193

employee_number	Suma de monthly_income	Alerta Salarial
1	5993	● Crítico
23	15427	● Crítico
27	3407	● Crítico
35	6825	● Crítico
38	18947	● Crítico
42	2086	● Crítico
47	2683	● Crítico
49	2014	● Crítico
52	5376	● Crítico
56	8726	● Crítico
60	4568	● Crítico
62	5772	● Crítico
68	5454	● Crítico
Total	3103791	● Crítico

- **Pitfall:** Confundir **seguridad de página** con **seguridad de datos**; ocultar una página no impide que alguien acceda a los **datos subyacentes**.
 - **El peligro:** Ocultar una página solo la esconde de la navegación inferior, pero cualquier usuario con **permisos de lectura (Viewer)** puede acceder a los datos subyacentes a través de:
 - **Informes personalizados (personalize visuals).**
 - **Exportación de datos a Excel.**
 - **Uso de "Analizar en Excel" (Analyze in Excel).**
 - **La solución:** La **seguridad a nivel de fila (RLS)** es el único mecanismo robusto para asegurar que los datos no sean accesibles, independientemente de cómo se visualice el reporte.

Desglose de Lógica (Code Blueprint)

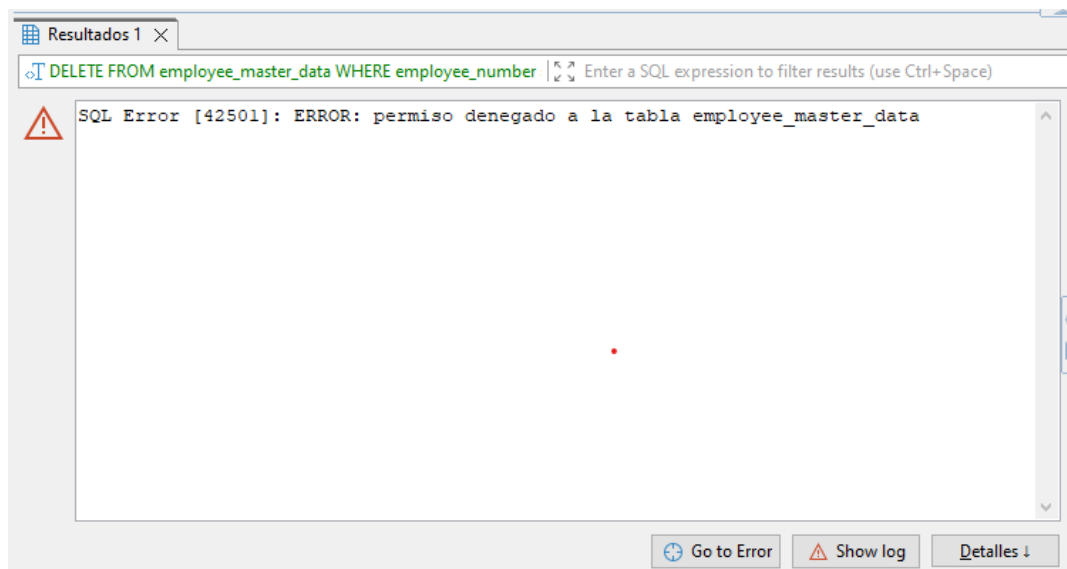
El Mindset: "Principio de Menor Privilegio"

El diseño de hoy sigue la **arquitectura de Zero Trust**:

- **Abstracción de Credenciales (Python)**: El **código** debe funcionar en cualquier máquina sin cambios, extrayendo la **configuración del entorno local**.
- **Jerarquía de Roles (SQL)**: Separamos al "Dueño del Dato" (DML - Data Manipulation Language / DDL - Data Definition Language) del "Consumidor del Dato" (Lectura).
- **Seguridad Granular (Power BI)**: Los **datos** son "inteligentes"; saben quién los mira y se **filtran automáticamente**.

Control de Calidad y Validación (El "Check")

- **Secret Leak Audit**: Verifica que tu archivo **.env** aparezca en **color gris** en **VS Code** (confirmando que **.gitignore** está funcionando).
- **Permission Test**: Intenta **borrar una fila** con el nuevo **rol de lectura**; el sistema debe **rechazar la operación**.
 - **Resultado PNG**: day05_delete_employee_record.png



- **RLS Simulation**: En **Power BI**, usa la función "**Ver como...**" para comprobar que los **filtros de seguridad** se aplican correctamente.

Micro-Proyecto (Hands-on Workflow): "The Vault & The Filter"

A. Fase SQL: Creación de Roles de Acceso (DBeaver)

Crearemos un **usuario específico** para **Power BI** que **no tenga permisos de escritura**.

/*****

Título: Configuración de Acceso para Power BI (HR Analytics)

Objetivo: Crear un usuario seguro y permisos de "solo lectura" para que Power BI se conecte a la base de datos de recursos humanos.

Descripción: Este script elimina configuraciones previas, crea un rol con permisos limitados, crea el usuario final y lo asigna a dicho rol.

Archivo SQL: day05_hr_analytics_pbi_readonly_permissions.sql

Archivo PNG: day05_hr_analytics_pbi_readonly_permissions_01.png

day05_hr_analytics_pbi_readonly_permissions_02.png

day05_hr_analytics_pbi_readonly_permissions_03.png

day05_hr_analytics_pbi_readonly_permissions_04.png

day05_hr_analytics_pbi_readonly_permissions_05.png

day05_hr_analytics_pbi_readonly_permissions_06.png

day05_hr_analytics_pbi_readonly_permissions_07.png

*****/

*/

-- 1. Limpieza inicial: Eliminamos el usuario si ya existía para evitar errores
-- de "usuario duplicado" al ejecutar el script de nuevo.

DROP USER IF EXISTS pbi_user;

-- 2. Creación del Rol: Creamos un rol (grupo de permisos) diseñado para
-- ser utilizado por la herramienta Power BI.

CREATE ROLE powerbi_reader;

-- 3. Permisos de Conexión: Permitimos al rol conectarse a la base de datos
-- específica de 'ibm_hr_analytics'.

GRANT CONNECTON DATABASE ibm_hr_analytics **TO** powerbi_reader;

-- 4. Permisos de Esquema: Damos permiso de uso (USAGE) sobre el
esquema 'public',

-- lo que permite al usuario ver la estructura, pero no modificarla.

GRANT USAGE ON SCHEMA public **TO** powerbi_reader;

-- 5. Permisos de Lectura: Otorgamos permiso para consultar (SELECT) datos

-- específicamente en la tabla 'employee_master_data'.

GRANT SELECT ON employee_master_data **TO** powerbi_reader;

-- 6. Creación del Usuario: Creamos el usuario 'pbi_user' con su contraseña.

CREATE USER pbi_user **WITH PASSWORD** 'SecurityPass2026';

-- 7. Asignación de Rol: Asignamos el rol creado anteriormente al usuario,

-- heredando así todos los permisos de lectura configurados.

GRANT powerbi_reader **TO** pbi_user;

- Resultado PNG: day05_hr_analytics_pbi_readonly_permissions_01.png

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,035s
Start time	Mon May 11 14:45:11 CST 2026
Finish time	Mon May 11 14:45:11 CST 2026
Query	-- 1. Limpieza inicial: Eliminamos el usuario si ya existía para evitar errores -- de "usuario duplicado" al ejecutar el script de nuevo. DROP USER IF EXISTS pbi_user

- Resultado PNG: day05_hr_analytics_pbi_readonly_permissions_02.png

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,006s
Start time	Mon May 11 14:46:08 CST 2026
Finish time	Mon May 11 14:46:08 CST 2026
Query	-- 2. Creación del Rol: Creamos un rol (grupo de permisos) diseñado para -- ser utilizado por la herramienta Power BI. CREATE ROLE powerbi_reader

- Resultado PNG: day05_hr_analytics_pbi_readonly_permissions_03.png

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,006s
Start time	Mon May 11 14:46:58 CST 2026
Finish time	Mon May 11 14:46:58 CST 2026
Query	-- 3. Permisos de Conexión: Permitimos al rol conectarse a la base de datos -- específica de 'ibm_hr_analytics'. GRANT CONNECT ON DATABASE ibm_hr_analytics TO powerbi_reader

- Resultado PNG: day05_hr_analytics_pbi_readonly_permissions_04.png

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,004s
Start time	Mon May 11 14:47:44 CST 2026
Finish time	Mon May 11 14:47:44 CST 2026
Query	-- 4. Permisos de Esquema: Damos permiso de uso (USAGE) sobre el esquema 'public', -- lo que permite al usuario ver la estructura, pero no modificarla. GRANT USAGE ON SCHEMA public TO powerbi_reader

- Resultado PNG: day05_hr_analytics_pbi_readonly_permissions_05.png

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,003s
Start time	Mon May 11 14:48:44 CST 2026
Finish time	Mon May 11 14:48:44 CST 2026
Query	-- 5. Permisos de Lectura: Otorgamos permiso para consultar (SELECT) datos -- específicamente en la tabla 'employee_master_data'. GRANT SELECT ON employee_master_data TO powerbi_reader

- **Resultado PNG: day05_hr_analytics_pbi_readonly_permissions_06.png**

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,056s
Start time	Mon May 11 14:49:28 CST 2026
Finish time	Mon May 11 14:49:28 CST 2026
Query	-- 6. Creación del Usuario: Creamos el usuario 'pbi_user' con su contraseña. CREATE USER pbi_user WITH PASSWORD 'SecurityPass2026'

- **Resultado PNG: day05_hr_analytics_pbi_readonly_permissions_07.png**

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,009s
Start time	Mon May 11 14:50:19 CST 2026
Finish time	Mon May 11 14:50:19 CST 2026
Query	-- 7. Asignación de Rol: Asignamos el rol creado anteriormente al usuario, -- heredando así todos los permisos de lectura configurados. GRANT powerbi_reader TO pbi_user

B. Fase Python: Ofuscación de Conexión (VS Code)

Refactorizaremos el script de ingesta para que sea 100% seguro.

```
# -----
#
# Título: Script de Validación de Conexión a PostgreSQL
#
# Objetivo: Establecer y verificar una conexión segura a una base de datos
# PostgreSQL
# utilizando SQLAlchemy.
#
# Descripción: Este script lee credenciales desde un archivo local (.env) para
# evitar
# exponer datos sensibles, configura el motor de conexión y realiza una
# prueba para
# asegurar que la base de datos es accesible.
```

```

#
# Archivo Python: day05_secure_main_connection.py
#
# Archivo PNG: day05_secure_main_connection.png
# -----
-----

# --- BLOQUE 1: Importación de herramientas ---
# pandas: Nos servirá para manejar tablas de datos más adelante
import pandas as pd
# sqlalchemy: Es el "traductor" que permite a Python hablar con la base de
datos
from sqlalchemy import create_engine
# os y dotenv: Se encargan de leer archivos de configuración secretos por
seguridad
import os
from dotenv import load_dotenv

# --- BLOQUE 2: Configuración del Entorno ---
# Carga las variables de entorno desde un archivo .env (usuarios, passwords,
etc.)
# Esto evita tener credenciales explícitas en el código.
load_dotenv()

# --- BLOQUE 3: Obtención de Credenciales y Validación ---
def get_secure_engine():
    """Recupera credenciales del sistema y verifica que estén completas."""
    # Lee cada credencial individual desde las variables de entorno
    db_user = os.getenv('DB_USER')
    db_pass = os.getenv('DB_PASS')
    db_host = os.getenv('DB_HOST')
    db_port = os.getenv('DB_PORT')
    db_name = os.getenv('DB_NAME')
    # Verifica que ninguna credencial esté vacía antes de intentar conectar
    if not all([db_user, db_pass, db_host, db_port, db_name]):
        raise ValueError("\n ✖ Error: Faltan credenciales en el archivo .env")

# --- BLOQUE 4: Creación del Motor (Engine) ---
# Crea el motor de SQLAlchemy (engine), que maneja el pool de
conexiones a la DB.
# Construye la URL de conexión usando el formato:
postgresql://user:pass@host:port/dbname
engine = create_engine(
    f"postgresql://{db_user}:{db_pass}@{db_host}:{db_port}/{db_name}"
)

```

```

    return engine

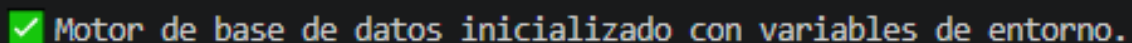
# --- BLOQUE 5: Inicialización y Prueba de Conexión ---
# Genera el motor de conexión llamando a la función definida anteriormente
engine = get_secure_engine()

def test_connection():
    """
    Intenta conectarse a la base de datos para verificar que el pipeline está
    listo para trabajar.
    """
    try:
        # Intenta abrir una conexión para confirmar que funciona
        with engine.connect() as conn:
            print("\n✅ Motor de base de datos inicializado con variables de
            entorno.")
            print("\n")
    except Exception as e:
        # Si ocurre un error, imprime el mensaje para diagnosticar el fallo
        print(f"\n❌ Error de Conexión: {e}")
        print("\n")

# --- BLOQUE 6: Ejecución Principal ---
if __name__ == "__main__":
    # Llama a la función de prueba solo si el script se ejecuta directamente.
    test_connection()

```

- Resultado PNG: day05_secure_main_connection.png



C. Fase Power BI: Implementación de RLS (Seguridad a Nivel de Fila)

Haremos que el **reporte** sea **dinámico** según el **departamento del usuario**.

- En Power BI, ve a la pestaña **Modelado** -> **Administrar roles**.



- Haz clic en **Crear** y nombra al rol: Gerente Ventas.

- En la tabla employee_master_data, añade el filtro DAX:
[department] = "Sales"

Administrar roles de seguridad

Cree nuevos roles de seguridad y use filtros para definir las restricciones de datos de nivel de fila.

Funciones

+ Nuevo

Gerente Ventas

Tablas

public employ...

Reglas

+ Nuevo ☒ Seleccionar todo Eliminar Agrupar Desagrupar

Mostrar datos si **Todas** de estas reglas son verdaderas

Columna	Condición	Valor
☐ department	Es Igual A	Sales

+ Nuevo

Guardar **Cerrar**

Cambiar al editor DAX

- **Crea** otro rol llamado: Gerente HR:
- En la tabla employee_master_data, añade el filtro DAX:
[department] = "Human Resources"

Administrar roles de seguridad

Cree nuevos roles de seguridad y use filtros para definir las restricciones de datos de nivel de fila.

Funciones

+ Nuevo

Gerente HR

Gerente Ventas

Tablas

public employ...

Reglas

+ Nuevo ☒ Seleccionar todo Eliminar Agrupar Desagrupar

Mostrar datos si **Todas** de estas reglas son verdaderas

Columna	Condición	Valor
☐ department	Es Igual A	Human Resources

+ Nuevo

Guardar **Cerrar**

Cambiar al editor DAX

- **Validación:** Ve a "Ver como...", selecciona Gerente_Sales y confirma que el dashboard solo muestra los registros de Ventas.

- **Resultado PNG: day05_sales_mgr_pbi_role.png**

employee_number	Primera fecha: department	Primera fecha: job_role	Suma de monthly_income
38	Sales	Manager	18947
323	Sales	Manager	19517
329	Sales	Manager	19068
558	Sales	Manager	18041
992	Sales	Manager	17444
1029	Sales	Manager	18303
1038	Sales	Manager	19845
1045	Sales	Manager	17650
1124	Sales	Manager	17465
1204	Sales	Manager	17048
1267	Sales	Manager	18213
1277	Sales	Manager	18824
Total	Sales	Manager	353193

employee_number	Suma de monthly_income	Alerta Salarial
1	5993	● Crítico
23	15427	● Crítico
27	3407	● Crítico
35	6825	● Crítico
38	18947	● Crítico
42	2086	● Crítico
47	2683	● Crítico
49	2014	● Crítico
52	5376	● Crítico
56	8726	● Crítico
60	4568	● Crítico
62	5772	● Crítico
68	5454	● Crítico
Total	3103791	● Crítico

- **Resultado PNG: day05_hr_mgr_pbi_role.png**

employee_number	Primera fecha: department	Primera fecha: job_role	Suma de monthly_income
140	Human Resources	Manager	18844
148	Human Resources	Manager	17328
644	Human Resources	Manager	18200
731	Human Resources	Manager	19141
734	Human Resources	Manager	19189
1338	Human Resources	Manager	19717
1625	Human Resources	Manager	19658
1973	Human Resources	Manager	19636
Total	Human Resources	Manager	151713

employee_number	Suma de monthly_income	Alerta Salarial
103	5021	Crítico
133	2073	Crítico
140	18844	Crítico
148	17328	Crítico
177	2942	Crítico
184	6347	Crítico
321	2267	Crítico
424	6410	Crítico
470	2696	Crítico
566	2564	Crítico
590	9950	Crítico
608	2741	Crítico
644	18200	Crítico
Total	419234	Crítico

Sesión de Mentoría 1:1 (Resolución)

- **Estrategia de Solución:** La forma más profesional de gestionar la seguridad es mediante **Service Accounts**. Nunca uses tu **usuario personal de base de datos** para conectar **Power BI**; usa el usuario **pbi_user** que creamos hoy.
- **Business Insights:** "La **seguridad de datos** protege la **reputación** de la empresa. Implementar **RLS (Row-Level Security)** asegura que la **información sensible de salarios** solo sea accesible por los **gerentes** correspondientes, evitando **conflictos internos** y cumpliendo con la **GDPR (General Data Protection Regulation) /LOPD (Ley Orgánica de Protección de Datos)**".
- **Debrief de Errores (Top 3):**
 - **Plain Text Passwords:** Dejar la **contraseña** escrita en el **script**. (Solución: Usar siempre **.env**).

- **Circular Permissions:** Dar permisos de **lectura** pero no de **USAGE** sobre el **esquema**. (**SQL** necesita ambos para funcionar).
- **Testing en Producción:** Probar los **roles** con **datos reales** sin antes validar en una **tabla de test**.



Checklist Operativa: Semana 1 - Día 5

- ☐ **SQL:** Creado el rol **powerbi_reader** y el usuario **pbi_user**.
- ☐ **Python:** **Script refactorizado** para usar **os.getenv** exclusivamente.
- ☐ **Git:** Verificado que el archivo **.env no** está en el **área de preparación (staged)**.
- ☐ **PBI:** Implementado y probado el **RLS** para al menos dos **departamentos**.
- ☐ **General:** Realizado el **Master Commit** de cierre de la **Semana 1**.

¿Lograste que el usuario **pbi_user** se conectara a la base de datos pero fallara al intentar ejecutar un **DELETE**?